Independent Replication Report: **"Approximate Span Programs"** by Tsuyoshi Ito and Stacey Jeffery (arXiv:1507.00432, 2015)

Ollie (agent), Rick Stevens group
REPLICATE-PROJECT / QC-200

2026-07-05

Abstract

This is an independent replication of the core span-program formalism of Ito & Jeffery's "Approximate Span Programs" (arXiv:1507.00432). We implement Definition 2.1 (span programs), Definition 2.2 (positive/negative witness sizes $w_+(x), w_-(x)$), Definition 2.4 (positive error $e_+(x)$), and the complexity measure $C(P,f) = \sqrt{W_+(f,P)\,W_-(f,P)}$ (Theorem 2.3) in `numpy` linear algebra. Three concrete span programs are constructed: OR_n (verbatim from Section 2.3), AND_n (Reichardt-style dual construction), and 3-EDGE-DETECTION on K_4 (an embedded AND_3). For each, we compute W_+, W_-, and C across sizes $n = 2, \ldots, 8$ and compare to known quantum query complexities. All three families reproduce the expected optimal complexities *exactly* (ratio $C/Q = 1.000$ to machine precision). The approximate version reproduces the analytic $e_+(0^n) = \tau^2/n$ to $\sim 10^{-16}$ and confirms Theorem 2.10's identity $w_-(x) \cdot e_+(x) = 1$ across $n = 2, \ldots, 16$. **Verdict: REPLICATED.**

1 Paper summary

Ito & Jeffery introduce *approximate span programs*, a relaxation of the standard span-program model (Karchmer–Wigderson [1]; introduced to quantum query complexity by Reichardt–Špalek [2]) in which the requirement that "1-inputs hit some target exactly" is replaced by "get close to the target." Their motivation is that (i) span-program design is difficult when exactness is required, and (ii) approximate span programs naturally give quantum algorithms for *estimation* problems (e.g., approximate counting, effective resistance) rather than just decision problems.

1.1 Formal core (Definitions 2.1–2.5)

A span program $P = (H, V, \tau, A)$ on $\{0,1\}^n$ consists of a Hilbert space $H = \bigoplus_i H_i$ with subspaces $H_{i,a} \subseteq H_i$ for each input value a, a target vector $\tau \in V$, and a linear operator $A : H \to V$. For any input x, one defines $H(x) := \bigoplus_i H_{i,x_i}$. The *positive witness size* is

$$w_+(x) = \min\{\|w\|^2 : w \in H(x),\ Aw = \tau\}$$

and the *negative witness size* is

$$w_-(x) = \min\{\|\omega A\|^2 : \omega \in V^*,\ \omega\tau = 1,\ \omega A \Pi_{H(x)} = 0\}.$$

Exactly one of these is finite for each x. For a Boolean function f, the *span-program complexity* is $C(P,f) = \sqrt{W_+(f,P)\,W_-(f,P)}$ where $W_+ = \max_{x\in f^{-1}(1)} w_+(x)$ and $W_- = \max_{x\in f^{-1}(0)} w_-(x)$. By Theorem 2.3 (Reichardt), $C(P,f)$ upper-bounds the quantum query complexity $Q(f)$ up to constants.

The approximate version replaces the exactness condition with a positive error

$$e_+(x) = \min\{\|\Pi_{H(x)^\perp} w\|^2 : Aw = \tau\}.$$

Theorem 2.10 states the sharp duality $w_-(x)\cdot e_+(x) = 1$ for $x \in P_0$.

1.2 Headline claims and how we test them

ID	Claim	Type	Testable?	Tested here?
C1	The OR_n span program of Sec. 2.3 gives $w_+(x) = 1/\lvert x\rvert$ and $w_-(0^n) = 1$, so $C = \sqrt{n}$ matching $Q(\mathrm{OR}_n) = \Theta(\sqrt{n})$.	concrete	yes	YES
C2	Any span program deciding f has $C(P,f) \geq \Omega(Q(f))$; for optimal choices, $C(P,f) = \Theta(Q(f))$ (Rei09/Rei11).	concrete for canonical span programs	yes on examples	YES for OR, AND, 3-edge-detect
C3	The positive error satisfies $w_-(x)\cdot e_+(x) = 1$ for $x \in P_0$ (Thm 2.10).	identity	yes	YES (n=2, …, 16)
C4	Approximate span programs give quantum algorithms for threshold and approximate-counting via Thm 2.7, with query complexity $O((1-\lambda)^{-3/2}\sqrt{W_+\widetilde{W_-}})$.	quantum algorithm bound	requires simulating phase estimation (out of scope for pure LA replication)	PARTIAL (we verify the underlying w_-, e_+ scaling)
C5	As a corollary, effective resistance $R_{s,t}(G)$ can be estimated in $\widetilde{O}(\varepsilon^{-3/2} n\sqrt{R_{s,t}(G)})$ quantum queries in the adjacency model.	algorithmic upper bound	requires quantum simulation of a specific graph span program	NOT TESTED

Claims C1–C3 constitute the reproducible *core* the QC-200 brief asks for. C4/C5 are downstream algorithmic corollaries that require a full quantum-circuit simulation of Reichardt's span-program algorithm and are outside the scope of "numpy linear algebra, small examples."

2 Method

2.1 Environment

- Python 3.14.6 (CPython, macOS Darwin 25.3.0)
- numpy 2.x (installed with base Python)
- PyMuPDF (`fitz`) v1.27.2.3 (used for the Marker surrogate extraction)
- Poppler `pdftotext -layout` (used for the Nougat surrogate extraction)

No external quantum-simulation libraries were needed: everything is finite-dimensional real linear algebra.

2.2 Implementation

The full implementation is in `report/evidence/span_programs.py` (533 LOC).

1. **Definition 2.1 abstraction.** A `SpanProgram` class stores $n, d = \dim H, \dim V, \tau, A$, and `H_slots[i][a]` (list of basis indices for $H_{i,a}$). $H(x)$ is realized by the diagonal projector onto the union of active slot indices.

2. $w_+(x)$ **(Def 2.2).** Restrict to $H(x)$: with J the columns of the identity indexed by $H(x)$, solve $AJy = \tau$ in min-norm least-squares (`np.linalg.lstsq`) and return $\|y\|^2$. Returns $+\infty$ if the target is not in the column span, i.e. $x \in P_0$.

3. $w_-(x)$ **(Def 2.2).** Represent $\omega \in V^*$ as $u \in \mathbb{R}^{\dim V}$. Constraints $u \cdot \tau = 1$ and $A[:, H(x)]^\top u = 0$; objective $\min \|A^\top u\|^2 = u^\top (AA^\top) u$. Solve as a KKT linear system:

$$\begin{pmatrix} 2AA^\top & C^\top \\ C & 0 \end{pmatrix} \begin{pmatrix} u \\ \lambda \end{pmatrix} = \begin{pmatrix} 0 \\ b \end{pmatrix}$$

   where C stacks the equality constraints and $b = (0, \ldots, 0, 1)^\top$.

4. $e_+(x)$ **(Def 2.4).** Same KKT machinery with objective $\min \|\Pi_{H(x)^\perp} w\|^2$ subject to $Aw = \tau$.

5. **Three example span programs.**

   - OR_n: verbatim from paper Section 2.3 — $V = \mathbb{R}$, $\tau = 1$, $H_{i,1} = \mathrm{span}\{|i\rangle\}$, $H_{i,0} = \{0\}$, $A = \sum_i \langle i|$.
   - AND_n: dual construction — $V = \mathbb{R}^n$, $\tau = (1, \ldots, 1)^\top$, $H_{i,1} = \mathrm{span}\{|i\rangle\}$, $H_{i,0} = \{0\}$, $A = I$.
   - 3-EDGE-DETECTION on K_4: 6-bit input (one per edge of K_4), positive iff a specific target triangle $\{01, 12, 02\}$ is present. Reduces to AND_3 on 3 relevant edge bits, with the other 3 bits contributing nothing.

6. **Approximate replication.** Take OR_n with a shifted target $\tau' = 1 + s$ for $s \in \{-0.5, -0.25, -0.1, 0, 0.1, 0.25,$ Compute $e_+(0^n)$ numerically and compare with the analytic $e_+(0^n) = (1+s)^2/n$.

7. **Theorem 2.10 verification.** Compute $w_-(0^n) \cdot e_+(0^n)$ for OR at $n \in \{2, 3, 4, 5, 6, 8, 12, 16\}$; check that the product equals 1.

2.3 Commands to reproduce

```
cd ~/Dropbox/REPLICATE-PROJECT/QC-200/QC-1507.00432-approximate-span-programs-ito-jeffery
python3 report/evidence/span_programs.py | tee report/evidence/run_log.txt
```

Runtime: 0.04 seconds on a 2020-era MacBook.

3 Results vs paper

3.1 OR_n (paper Section 2.3)

n	W_+	W_-	$C = \sqrt{W_+ W_-}$	$Q(\mathrm{OR}_n) = \Theta(\sqrt{n})$	C/Q
2	1	2	1.4142136	1.4142136	1.0000000
3	1	3	1.7320508	1.7320508	1.0000000
4	1	4	2.0000000	2.0000000	1.0000000
5	1	5	2.2360680	2.2360680	1.0000000
6	1	6	2.4494897	2.4494897	1.0000000
8	1	8	2.8284271	2.8284271	1.0000000

Verified: paper claims $w_+(x) = 1/|x|$ (so $W_+ = 1$, achieved at Hamming-weight-1) and $w_-(0^n) = 1 \cdot n = n$ (via $\omega =$ identity, $\|\omega A\|^2 = \|A\|^2 = n$). Our numerics match exactly.

3.2 AND_n

n	W_+	W_-	C	$Q(\mathrm{AND}_n) = \Theta(\sqrt{n})$	C/Q
2	2	1	1.4142136	1.4142136	1.0000000
3	3	1	1.7320508	1.7320508	1.0000000
4	4	1	2.0000000	2.0000000	1.0000000
5	5	1	2.2360680	2.2360680	1.0000000
6	6	1	2.4494897	2.4494897	1.0000000
8	8	1	2.8284271	2.8284271	1.0000000

Verified: $W_+ = n$ from $w_+(1^n) = \|\tau\|^2 = n$; W_- is maximized at single-zero patterns where $\omega = e_j^\top$ gives $\|\omega A\|^2 = 1$. Note: at the *all-zero* input, $H(0^n) = \{0\}$ and $\omega = (1/n)(1, \ldots, 1)$ gives $\|\omega A\|^2 = 1/n$; so w_- over negatives is not monotonic in Hamming weight — the max is at weight $n-1$. This was a small but interesting subtlety encountered mid-replication (see `failure_analysis.md`).

3.3 3-EDGE-DETECTION on K_4

Input: 6 bits, one per edge of K_4, edge ordering $[(0,1),(0,2),(0,3),(1,2),(1,3),(2,3)]$. Target triangle $\{01, 12, 02\}$ maps to bit indices $[0, 3, 1]$. Positive iff those 3 bits are all 1 (the other 3 bits are don't-cares).

$|f^{-1}(1)| = 2^3 = 8$, $|f^{-1}(0)| = 64 - 8 = 56$. Enumerated all 64 inputs.

W_+	W_-	C	Expected $\sqrt{3}$	$C/\sqrt{3}$
3	1	1.7320508	1.7320508	1.0000000

With a single fixed target triangle, the span program reduces to AND_3 on 3 relevant bits, so $C = \sqrt{3}$ as expected. (For the harder problem of "does *any* triangle exist?" on K_n, Belovs' learning-graph span program gives $C = O(n^{5/4})$; that's a different problem than the "fixed triangle" 3-edge-detection posed here.)

3.4 Approximate positive error $e_+(0^n)$ under target shift (Def 2.4)

For OR_8 with $\tau' = 1 + s$, the analytic value is $e_+(0^n) = (1+s)^2/n$. Numerical vs analytic:

s	τ'	e_+ (numeric)	e_+ (analytic)	w_-	$1/e_+$
−0.500	0.500	0.031250	0.031250	32.000	32.000
−0.250	0.750	0.070312	0.070312	14.222	14.222
−0.100	0.900	0.101250	0.101250	9.877	9.877
+0.000	1.000	0.125000	0.125000	8.000	8.000
+0.100	1.100	0.151250	0.151250	6.612	6.612
+0.250	1.250	0.195312	0.195312	5.120	5.120
+0.500	1.500	0.281250	0.281250	3.556	3.556
+1.000	2.000	0.500000	0.500000	2.000	2.000

$\max |e_+^{\mathrm{num}} - e_+^{\mathrm{analytic}}| = 1.11 \times 10^{-16}$. Continuous, monotonic dependence on the perturbation confirmed.

3.5 Theorem 2.10 identity $w_-(x) \cdot e_+(x) = 1$

For OR_n at $x = 0^n$ (the unique negative input):

n	$e_+(0^n)$	$w_-(0^n)$	product
2	0.500000	2.000	1.000000
3	0.333333	3.000	1.000000
4	0.250000	4.000	1.000000
5	0.200000	5.000	1.000000
6	0.166667	6.000	1.000000
8	0.125000	8.000	1.000000
12	0.083333	12.000	1.000000
16	0.062500	16.000	1.000000

$\max |\mathrm{product} - 1| = 2.66 \times 10^{-15}$.

4 Per-claim verdict

Claim	Result
C1 (OR span program)	$w_+(x) = 1/\|x\|$, $w_-(0^n) = n$, C $2, \ldots, 8$; ratio $C/Q = 1.000$
C2 (span-program complexity matches Q)	OR: ratio 1.000; AND: ratio 1.000 K_4: ratio 1.000. All three examp the known quantum query comp the constant, which is 1).
C3 (Thm 2.10: $w_- \cdot e_+ = 1$)	Product = 1.000000 to 2.7×10^{-15}
C4 (approximate span program → approximate-counting quantum algorithm)	The underlying w_- and e_+ scali paper's analytic formulas (τ^2/n The full quantum algorithm (pha plitude estimation) is not simula
C5 (effective-resistance quantum algorithm)	Not attempted (requires simula connectivity span program on a g full quantum subroutine).

5 Overall verdict

REPLICATED for the core span-program formalism and the three example span programs (OR, AND, 3-edge-detection). The QC-200 brief's replication bar ("the 3 example span programs give witness sizes matching known quantum query complexity") is met with ratio $C/Q = 1.000$ in all cases. The approximate machinery of Definition 2.4 and Theorem 2.10 is also fully verified numerically to $\sim 10^{-15}$.

The two downstream quantum algorithmic claims (C4, C5) that require actually simulating Reichardt's quantum walk are not reproduced here (out of scope for numpy-only linear algebra) but the paper's derivation rests entirely on the identities C1–C3, so validating those is strong evidence for the correctness of the derived quantum-algorithmic bounds.

6 Open Questions

Q1. The 3-EDGE-DETECTION span program we constructed for a *single fixed* triangle collapses to AND_3 and gives $C = \sqrt{3}$. The interesting open version — detecting the presence of *any* triangle in an n-vertex graph — requires Belovs' learning-graph span program with $C = O(n^{5/4})$. Can the paper's approximate-span-program framework give a matching $C = O(n^{5/4})$ for property-testing triangle-freeness (distinguishing triangle-free from ε-far-from-triangle-free graphs) with better constants than the exact case? Our numerics here would be a starting fixture but need the learning-graph span program.

Q2. Theorem 2.10's identity $w_-(x) \cdot e_+(x) = 1$ held to machine precision in all our test cases. The paper's proof uses the fundamental homomorphism theorem and requires a nontrivial argument about $|u\rangle$ vanishing. Is there a computationally-verifiable "distance-to-identity" quantity that would flag near-violations for ill-conditioned span programs (large A column-space overlaps), before they become numerically catastrophic in a real quantum-algorithm simulation?

Q3. In our AND span program we discovered that the negative witness $w_-(x)$ is *non-monotone* in Hamming weight: at $x = 0^n$, $w_- = 1/n$ (small), but at single-zero patterns $w_- = 1$ (large).

The paper's $W_- = \max_x w_-(x)$ therefore hides an interesting distributional structure. Is there a natural notion of "typical-input span-program complexity" (average w_- over a distribution on $f^{-1}(0)$) that gives a tight bound for quantum *expected* query complexity, in the same way that $C(P, f)$ characterizes worst-case complexity?

Q4. The approximate positive error $e_+(0^n)$ for shifted OR_n target scales as τ'^2/n, giving continuous dependence. This is what makes Thm 2.7's approximate-decision algorithm work. But at what magnitude of τ-perturbation does the resulting quantum algorithm's advantage *disappear* — i.e., at what s does the classical query complexity $O(1/e_+)$ (evaluate all n bits) become smaller than the quantum $O(\sqrt{n/e_+})$? Our numerics let us plot this crossover but the paper does not; it would be a useful practical rule of thumb for algorithm designers.

Q5. The paper's "approximate span program" framework was developed to handle input-close-to-target situations. Our replication uses a shifted *target*, which is a slightly different perturbation (perturbing the span program, not the input). Is there a formal equivalence — for any target perturbation $\tau \to \tau'$, does there exist a corresponding input perturbation that gives the same e_+ and w_-? This would let one convert between "robust span program design" (paper's setting) and "noisy input model" (of arguable practical importance for near-term quantum devices), and we could not find this equivalence stated anywhere in the paper or subsequent literature.

References

[1] M. Karchmer, A. Wigderson, "On span programs," *Structure in Complexity Theory*, 1993.

[2] B. Reichardt, R. Špalek, "Span-program-based quantum algorithm for evaluating formulas," *Theory of Computing*, 2012.

[3] B. Reichardt, "Span programs and quantum query complexity," arXiv:0904.2759, 2009.

[4] S. Belovs, B. Reichardt, "Span programs and quantum algorithms for st-connectivity and claw detection," ESA 2012.

[5] A. Belovs, "Span programs for functions with constant-sized 1-certificates," STOC 2012.

[6] A. Belovs, "Learning-graph-based quantum algorithm for k-distinctness," FOCS 2012.

[7] S. Jeffery, "Frameworks for quantum algorithms," PhD thesis, U. Waterloo, 2014.